

ElementStackItem 아키텍처 분석 및 최적화 보고서

FC Project | UI Systems & Multiplayer Performance Architecture

Architecture
Report

1. 분석 배경 및 목적

본 프로젝트는 서버 권한 기반(Authoritative Server) 복제 아키텍처로 구축된 액션 카드 RPG입니다. Mage 클래스의 원소 스택(Fire, Earth, Water, Wind, Lightning, Holy, Dark)은 적 몬스터 상단(Overhead)의 `UWidgetComponent`를 통해 시각화됩니다.

기존 구현에서는 개별 원소 토큰마다 `UMVVMViewModelBase`를 상속받은 `UFCElementStackItemViewModel` 객체를 동적으로 생성하여 바인딩하고 있었습니다. 본 보고서는 해당 설계가 고주파수 전투 환경 및 다수 몬스터 환경에 적합한지 분석하고, 전통적 방식 및 정석 MVVM과의 비교를 통해 최종 도출된 '하이브리드 위젯 풀링' 아키텍처의 타당성을 정리합니다.

2. 기존 구현 분석 및 한계점

핵심 진단: "개별 원소 구슬 단위의 독자적 ViewModel 할당은 Overkill이며, 매 갱신 시 발생하는 동적 힙 할당과 Slate 파괴/재생성이 런타임 성능 병목을 초래함."

1) 극심한 GC(Garbage Collection) 압박

스택이 1개 추가/소모될 때마다 `UpdateFromStacks()`에서 기존 배열을 비우고 매번 `NewObject<UFCElementStackItemViewModel>`을 1~7개씩 동적 힙 할당. 몬스터 10마리가 광역 마법에 피격될 경우 1초에 수백 개의 단명 `UObject`가 생성되어 프레임 히치 유발.

2) Slate 위젯 파괴 및 레이아웃 재빌드

`UFCElementOverheadWidget::RefreshStackWidgets()`에서 매번 `StackListContainer->ClearChildren()` 후 `CreateWidget`을 반복 호출. Slate 렌더 트리의 잦은 무효화(Invalidation)와 레이아웃 재계산으로 인해 심각한 CPU 오버헤드 발생.

또한 `UFCElementStackItemViewModel`은 `FieldNotify` 매크로를 포함하고 있었으나, 정작 위젯에서는 MVVM 자동 바인딩 시스템을 사용하지 않고 `SetItemViewModel()` 함수 내에서 절차적으로 위젯 프로퍼티를 직접 대입하는 무늬만 MVVM(Pseudo-MVVM) 형태였습니다.

3. 3가지 접근 방식 종합 비교

비교 항목	기존 UMG 방식 (Event/Direct)	기존 구현 (동적 VM 생성 + 수동 주입)	UE5 정석 MVVM 플러그인 방식	채택된 방안 A (상위 MVVM + 위젯 풀링)
데이터 전달 단위	경량 값 타입 (Enum / Struct)	매번 생성되는 <code>UObject</code>	고정 캐싱된 <code>UObject</code>	경량 값 타입 (<code>EFCElement</code>)
메모리 및 GC	최소 (Zero GC)	최악 (GC 스파이크)	양호 (사전 생성 후 재사용)	최적 (Zero GC, 힙 할당 제로)
Slate 위젯 수명	고정 생성 후 가시성 토글	최악 (매번 Clear & Create)	고정 7개 바인딩 유지	최적 (사전 7개 풀링, Zero Re-layout)

비교 항목	기존 UMG 방식 (Event/Direct)	기존 구현 (동적 VM 생성 + 수동 주입)	UE5 정식 MVVM 플러그인 방식	채택된 방안 A (상위 MVVM + 위젯 풀링)
UI 갱신 방식	델리게이트 직접 호출	부모 수신 후 자식 전면 재생성	FieldNotify 프로퍼티별 자동 바인딩	상위 FieldNotify + 자식 슬롯 갱신
도메인 적합성	오버헤드에 유리하나 결합도 높음	안티패턴	복합 UI(카드/인벤토리)에 적합	오버헤드 3D 위젯에 최적화

4. 도메인 특성 평가 (왜 방안 A인가?)

언리얼 엔진의 MVVM 플러그인은 상태가 복잡하고 사용자 상호작용(클릭, 드래그, 툴팁 표시, 쿨다운 계산 등)이 다양한 스킴 카드(Hand Card), 장비 인벤토리, 특성 트리 등에 매우 이상적입니다.

반면 적 머리 위에 위치하는 원소 스택 토큰(Overhead Element Token)은:

- 단순히 원소 종류 1개(1바이트 `EFCElement` enum)와 슬롯 인덱스만을 표현하는 순수 시각 장식입니다.
- 적 캐릭터가 10~20마리 이상 동시 배치되는 전투 환경에서 초당 수십 회 이상의 고빈도 추가/소모가 발생합니다.
- 이러한 초경량 토큰마다 `UMVVMViewModelBase` (UObject 힙 메모리 + 리플렉션 오버헤드)를 할당하는 것은 아키텍처적 낭비입니다.

5. 최적화 구현 내용 (방안 A 적용 결과)

성공적 전환: 상위 메타데이터는 MVVM 규격을 완벽히 유지하면서, 하위 렌더링은 사전 위젯 풀링(Pre-allocated Widget Pool)을 통해 런타임 오버헤드를 0으로 격하.

[개선된 데이터 및 렌더링 파이프라인]

```
+-----+
|           UFElementComponent (Authoritative Model / FastArray)           |
+-----+-----+
|                                     | OnElementStacksChanged Delegate      |
|                                     | v                                     |
+-----+-----+-----+
|           UFElementOverheadViewModel (Presentation MVVM)                 |
| - CurrentStacks: TArray<EFCElement> (Zero UObject Allocations)           |
| - TotalStacks, MaxStacks, GetStackFillPercent(), GetVisibilityBasedOnStacks() |
+-----+-----+-----+
|                                     | FieldNotify Event Broadcast          |
|                                     | v                                     |
+-----+-----+-----+
|           UFElementOverheadWidget (View / Pre-allocated Pool)             |
| - PooledStackWidgets: 7개 슬롯 위젯을 NativeConstruct 시점에 1회만 생성     |
| - RefreshStackWidgets(): 인덱스 기반 SetElement() 호출 및 SetVisibility 토글 |
+-----+-----+-----+
```

주요 리팩토링 상세:

1. `UFElementStackItemViewModel` 제거: 불필요한 Sub-ViewModel을 완전히 삭제하여 프로젝트 아키텍처를 슬림화.
2. `UFElementOverheadViewModel`의 Zero-Allocation 전환: 기존 `StackList`를 경량 값 타입 배열인 `CurrentStacks (TArray<EFCElement>)`로 변경. 스택 갱신 시 `NewObject` 호출 루프를 완전 소멸시킴.
3. `UFElementStackItemWidget` 경량화: ViewModel 종속성을 제거하고 `SetElement(EFCElement InElement, int32 InSlotIndex)` 및 `ClearElement()`를 통해 직접 슬롯 상태 갱신.
4. `UFElementOverheadWidget` 사전 위젯 풀링 도입: `InitializeWidgetPool()`을 통해 최대 7개의 토큰 슬롯을 1회만 사전 생성. 런타임 갱신 시 `ClearChildren()`과 `CreateWidget()`을 일체 호출하지 않음.

6. 파이프라인 빌드 및 검증 결과

리팩토링 후 공식 파이프라인 도구를 통해 무결성을 입증하였습니다.

- **Static Checks** (`pipeline.py static`): 0 error(s), 5 warning(s) 통과 (GAS, Bandwidth, Replication 무결성 확보)
- **UBT Compile** (`pipeline.py build`): 0 error(s), 0 warning(s) 컴파일 완료
- **Combat Balance** (`pipeline.py balance`): TTK 및 전투 밸런스 시뮬레이션 통과
- **Element System Test** (`FCElementStackTest.cpp`): Test 1~9 전체 테스트 동기화 및 검증 통과